Name- Shreyash Khapekar

PRN - B25ET1019

Division- SY1 ENTC

Title: Perform Bubble Sort, Selection Sort, Insertion Sort, Quick Sort, and Merge Sort using C

```
#include <stdio.h>

void printArray(int arr[], int n){
   int i;
   for(i = 0; i < n; i++){
      printf("%d ", arr[i]);
   }
   printf("\n");
}

void copyArray(int src[], int dest[], int n){
   int i;
   for(i = 0; i < n; i++){
      dest[i] = src[i];
   }
}

void bubbleSort(int arr[], int n){
   int i, j, temp;
   for(i = 0; i < n - 1; i++){
      for(j = 0; j < n - i - 1; j++){
         if(arr[j] > arr[j+1]){
            temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
         }
      }
   }
}

void selectionSort(int arr[], int n){
   int i, j, minIdx, temp;
   for(i = 0; i < n - 1; i++){
```

```
        minIdx = i;
        for(j = i + 1; j < n; j++){
            if(arr[j] < arr[minIdx])
                minIdx = j;
        }
        temp = arr[minIdx];
        arr[minIdx] = arr[i];
        arr[i] = temp;
    }
}

void insertionSort(int arr[], int n){
    int i, key, j;
    for(i = 1; i < n; i++){
        key = arr[i];
        j = i - 1;
        while(j >= 0 && arr[j] > key){
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

int partition(int arr[], int low, int high){
    int pivot = arr[high];
    int i = low - 1;
    int j, temp;

    for(j = low; j < high; j++){
        if(arr[j] < pivot){
            i++;
            temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return (i + 1);
}

void quickSort(int arr[], int low, int high){
```

```
    if(low < high){
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

void merge(int arr[], int l, int m, int r){
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[50], R[50];

    for(i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for(j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    i = 0;
    j = 0;
    k = l;

    while(i < n1 && j < n2){
        if(L[i] <= R[j]){
            arr[k] = L[i];
            i++;
        }
        else{
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while(i < n1){
        arr[k] = L[i];
        i++;
        k++;
    }

    while(j < n2){
        arr[k] = R[j];
        j++;
```

```
        k++;
    }
}

void mergeSort(int arr[], int l, int r){
    if(l < r){
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main(){
    int arr[100], temp[100], n, choice, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter unsorted elements: ");
    for(i = 0; i < n; i++){
        scanf("%d", &arr[i]);
    }

    while(1){
        printf("\n1. Bubble Sort\n2. Selection Sort\n3. Insertion Sort\n4. Quick Sort\n5. Merge Sort\n6. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        copyArray(arr, temp, n);

        if(choice == 1){
            bubbleSort(temp, n);
            printf("Sorted array: ");
            printArray(temp, n);
        }
        else if(choice == 2){
            selectionSort(temp, n);
            printf("Sorted array: ");
            printArray(temp, n);
        }
        else if(choice == 3){
            insertionSort(temp, n);
            printf("Sorted array: ");
```

```
            printArray(temp, n);
        }
        else if(choice == 4){
            quickSort(temp, 0, n - 1);
            printf("Sorted array: ");
            printArray(temp, n);
        }
        else if(choice == 5){
            mergeSort(temp, 0, n - 1);
            printf("Sorted array: ");
            printArray(temp, n);
        }
        else if(choice == 6){
            break;
        }
        else{
            printf("Invalid Choice!\n");
        }
    }

    return 0;
}
```